

Panduan Lengkap Proyek IMS-DC-SDK

5G New Calling Terminal SDK - Dokumentasi Komprehensif Dikembangkan oleh China Telecom Research Institute

Daftar Isi

1. [Memulai Cepat](#)
2. [Apa Itu Proyek Ini?](#)
3. [Penjelasan Konsep Kunci](#)
4. [Gambaran Arsitektur](#)
5. [Cara Kerja - Alur End-to-End](#)
6. [Pembuatan Data Channel](#)
7. [Manajemen Lifecycle Panggilan](#)
8. [Struktur File & Organisasi Kode](#)
9. [Alur Fungsi Detail](#)
10. [Panduan Pengembangan](#)
11. [Sumber Daya Tambahan](#)

Memulai Cepat

Untuk Pengguna Non-Teknis

Baca: [Ringkasan Alur Sederhana](#) (5 menit) **Pahami:** Apa yang dilakukan SDK ini dan mengapa penting

Untuk Developer Mini App (HTML/JS)

Baca: [Konsep Kunci](#) → [Pembuatan Data Channel](#) **Waktu:** 30 menit **Output:** Memahami cara membuat aplikasi yang berjalan saat panggilan telepon

Untuk Developer Android SDK

Baca: Semua bagian secara berurutan **Waktu:** 2-3 jam **Output:** Pemahaman lengkap tentang arsitektur dan implementasi SDK

Apa Itu Proyek Ini?

Gambaran Besar

5G New Calling menambahkan **data channel** (IMS Data Channel) di atas IMS audio dan video channel tradisional, memungkinkan pertukaran informasi interaktif selama panggilan telepon dengan mengintegrasikan teknologi AR, AI, dan lainnya.

Panggilan Telepon Tradisional:

 [Voice] ----->  [Mendengar percakapan]
 [Video] ----->  [Melihat satu sama lain]

5G New Calling:

 [Voice] ----->  [Mendengar percakapan]
 [Video] ----->  [Melihat satu sama lain]
 [Data] ----->  [Berbagi menu, file, AR, game] ★ BARU!

Contoh Dunia Nyata

Skenario: Anda menelepon restoran untuk memesan makanan.

Sebelum (Panggilan Tradisional):

-  Anda: "Ada menu apa?"
-  Restoran: "Kami punya pizza, pasta, salad..." (*menjelaskan semuanya secara verbal*)

Sesudah (5G New Calling dengan IMS-DC-SDK):

-  Anda: "Ada menu apa?"
-  Restoran: "Saya kirimkan menunya!" (*mengirim menu interaktif via data channel*)
-  Layar Anda menampilkan: Menu lengkap dengan gambar, harga, dan tombol "Pesan"
- Anda tap item untuk memesan sambil tetap berbicara di telepon!

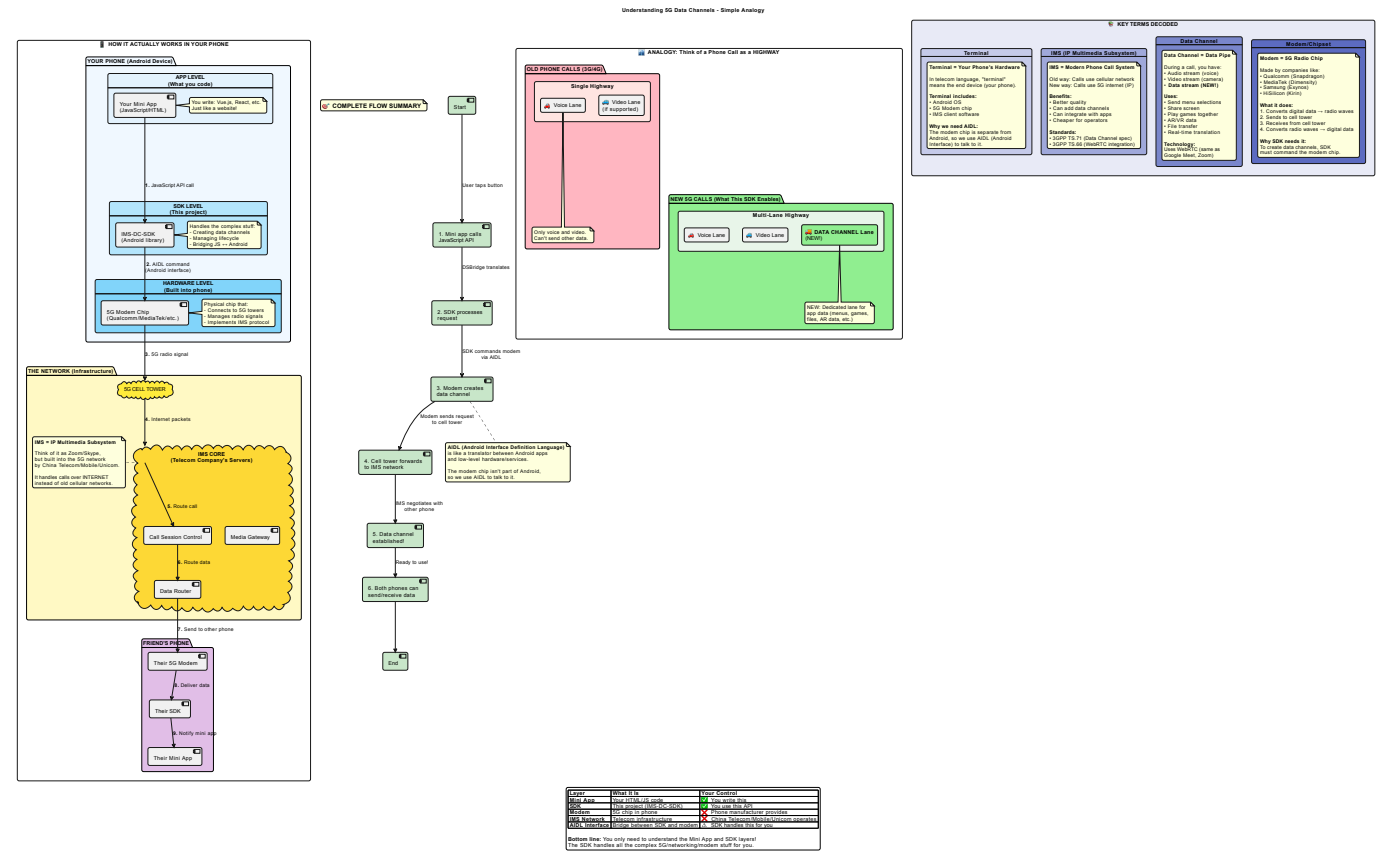
Apa yang Dilakukan SDK Ini

IMS-DC-SDK (5G New Calling Terminal SDK):

1. **Menyediakan Runtime Environment** untuk mini app (aplikasi HTML5/JavaScript)
2. **Menelola Lifecycle Data Channel** yang terikat dengan status panggilan telepon
3. **Menjembatani Komunikasi** antara mini app dan hardware 5G (modem)
4. **Memastikan Keamanan** dengan isolated storage dan manajemen permission
5. **Mendukung Multiple App** yang berjalan bersamaan selama panggilan

Penjelasan Konsep Kunci

Panduan Visual Konsep 5G



Terminologi Penting

Istilah	Penjelasan Sederhana	Definisi Teknis
Terminal	Smartphone Android Anda	Perangkat fisik dengan Android OS + chip modem 5G
IMS Network	Sistem panggilan telepon modern lewat internet	IP Multimedia Subsystem - menangani panggilan VoLTE/5G menggunakan packet-switched network
Modem	Chip 5G di dalam ponsel Anda	Komponen hardware yang menangani komunikasi seluler (Qualcomm, MediaTek, dll.)
Data Channel	Jalur data tambahan saat panggilan	Channel berbasis IMS untuk mengirim data aplikasi bersamaan dengan voice/video
AIDL	Jembatan antara Android dan modem	Android Interface Definition Language - memungkinkan komunikasi antar-process
Mini App	Aplikasi HTML5 yang berjalan saat panggilan	Aplikasi berbasis web (HTML/CSS/JS) yang dikelola oleh SDK
TS.71	Standar industri untuk data channel	Spesifikasi teknis yang mendefinisikan interface AIDL antara SDK dan terminal
WebRTC	Komunikasi real-time untuk web	Framework yang digunakan untuk membuat peer-to-peer data channel

Analogi Jalan Tol

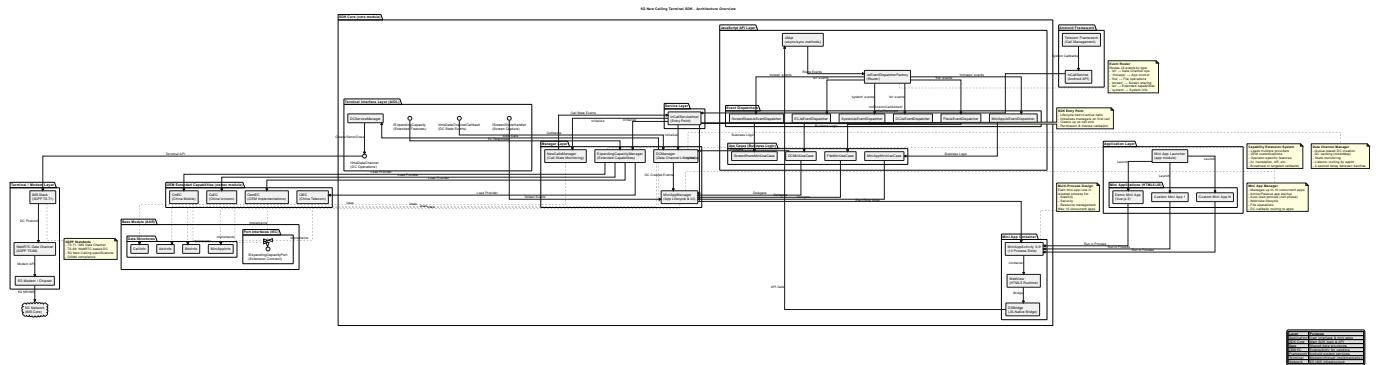
Bayangkan panggilan telepon seperti sistem jalan tol:

- | | | |
|---|-------------|--|
|  | Jalur Voice | - Membawa kata-kata yang Anda ucapkan |
|  | Jalur Video | - Membawa video stream (jika video call) |
|  | Jalur Data | - Membawa data aplikasi (menu, file, game) ★ BARU! |

Semua jalur berjalan bersamaan di "jalan tol 5G" yang sama (IMS network).

Gambaran Arsitektur

Diagram Arsitektur Sistem



Komponen Utama

1 Layer InCallService

- **Apa:** Inherit Android `InCallService` untuk memantau status panggilan
- **Mengapa:** Memastikan SDK hanya berjalan saat ada panggilan aktif
- **Lokasi:** `core/src/main/java/com/ct/ertclib/dc/core/service/InCallServiceImpl.kt`

2 Interface AIDL TS.71

- **Apa:** Interface standar antara SDK dan hardware terminal
- **Mengapa:** Memungkinkan SDK mengontrol data channel tanpa perlu tahu detail hardware
- **Lokasi:** `core/src/main/aidl/com/newcalllib/datachannel/V1_0`
- **Fungsi:**
 - Mendapatkan daftar mini app dari network
 - Download package mini app
 - Membuat/menghancurkan data channel
 - Mengirim/menerima data

3 Interface Extended Capability (IEC)

- **Apa:** Sistem plugin untuk fitur khusus operator/manufacturer
- **Mengapa:** Memungkinkan China Telecom (atau operator lain) menambahkan fitur unik sambil menjaga SDK tetap terpadu
- **Contoh:** Fitur AR khusus, API pembayaran khusus, layanan khusus operator
- **Lokasi:**
 - Interface: `base/src/main/java/com/ct/ertclib/dc/base/port/ec/IEC.kt`

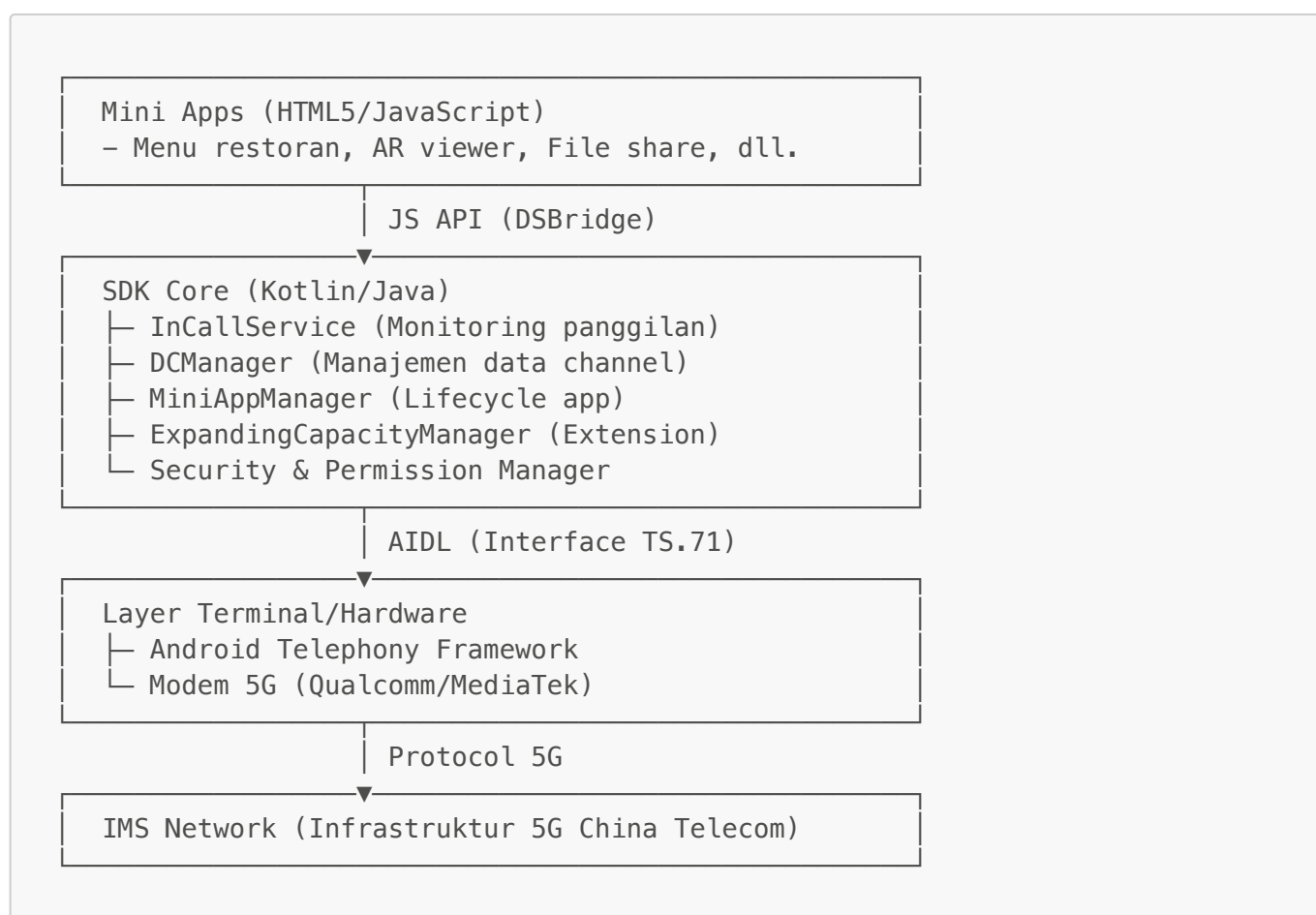
- Manager:
core/src/main/java/com/ct/ertclib/dc/core/manager/common/ExpandingCapacityManager.kt
- Contoh: oemec/src/main/java/com/ct/oemec/OemEC.kt

4 Layer JS API (DSBridge)

- **Apa:** Interface JavaScript yang diekspos ke mini app
- **Mengapa:** Developer mini app memanggil fungsi JS sederhana daripada kode Android/AIDL yang kompleks
- **Framework:** DSBridge
- **Lokasi:** core/src/main/java/com/ct/ertclib/dc/core/miniapp/bridge/JSApi.kt
- **Contoh API:**

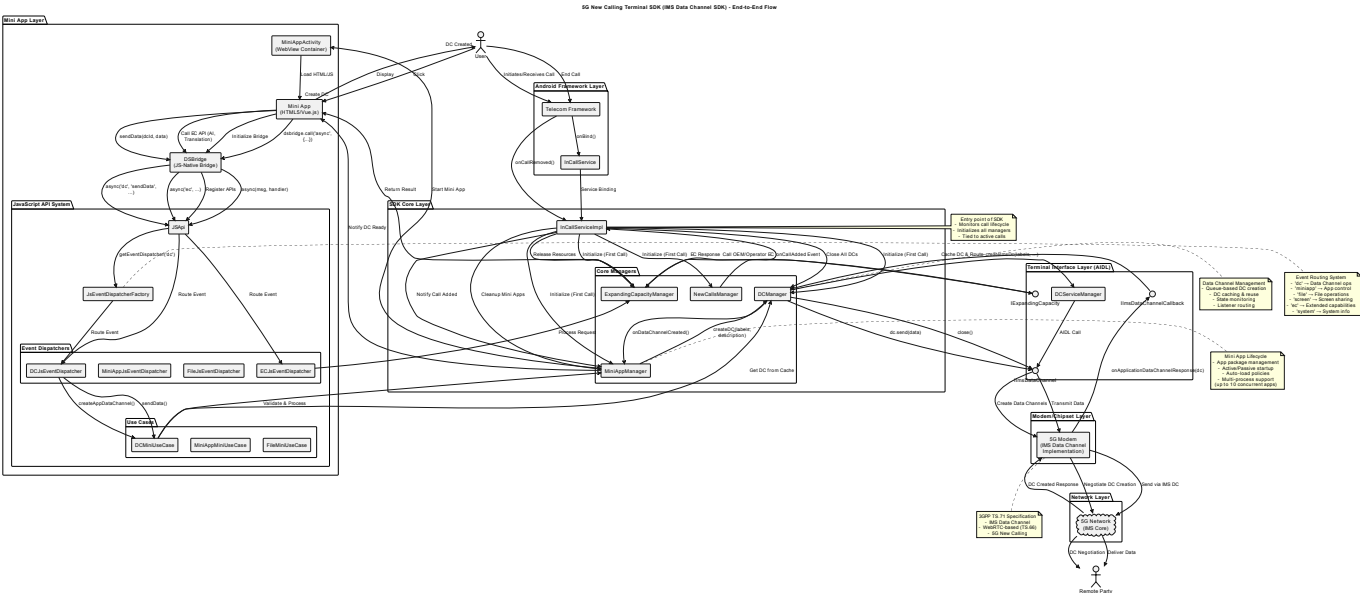
```
// Kode mini app
await bridge.createDataChannel({ channelId: 'menu-channel' })
await bridge.sendData({ data: menuSelection })
```

Layer Arsitektur



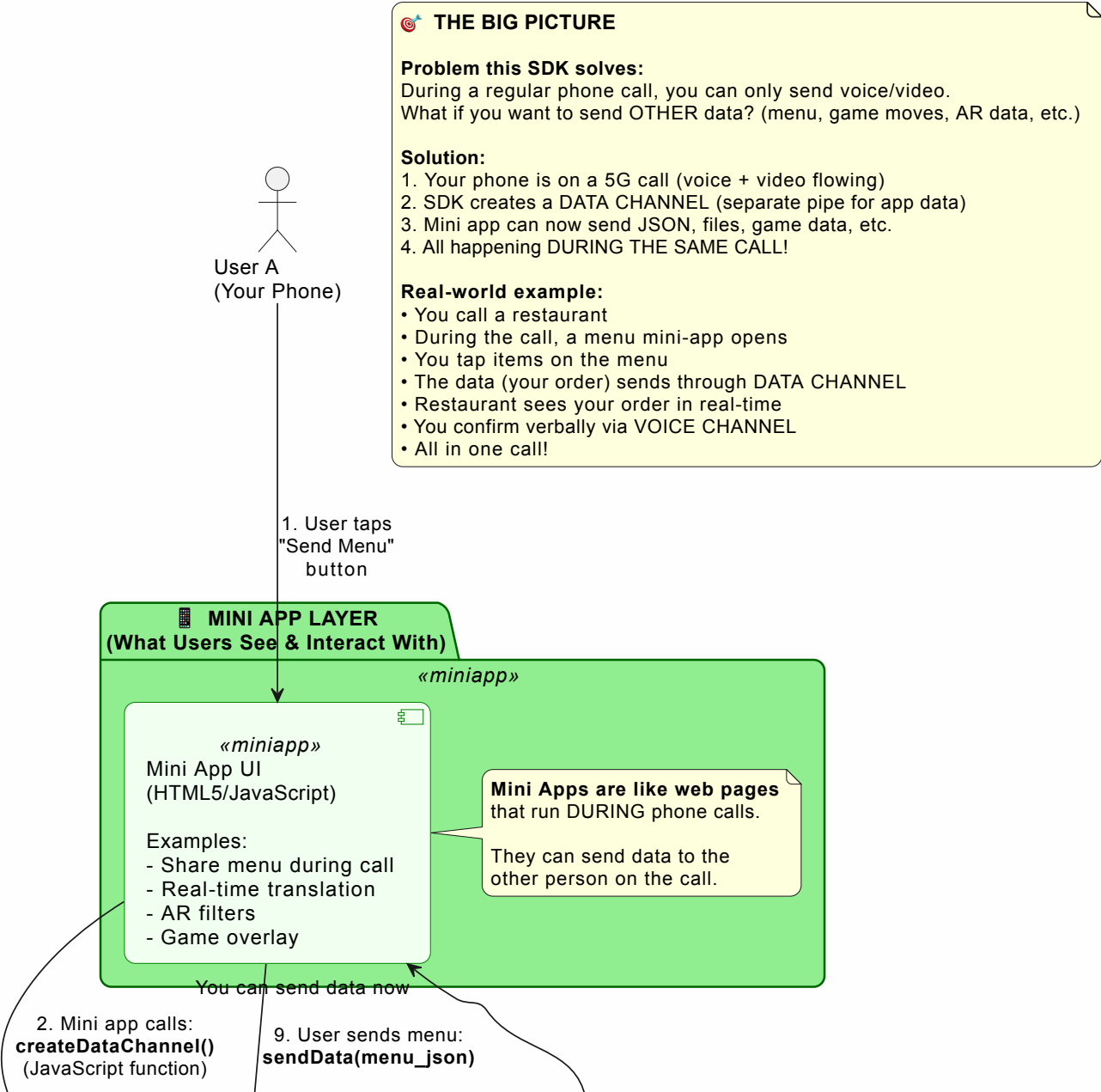
Cara Kerja - Alur End-to-End

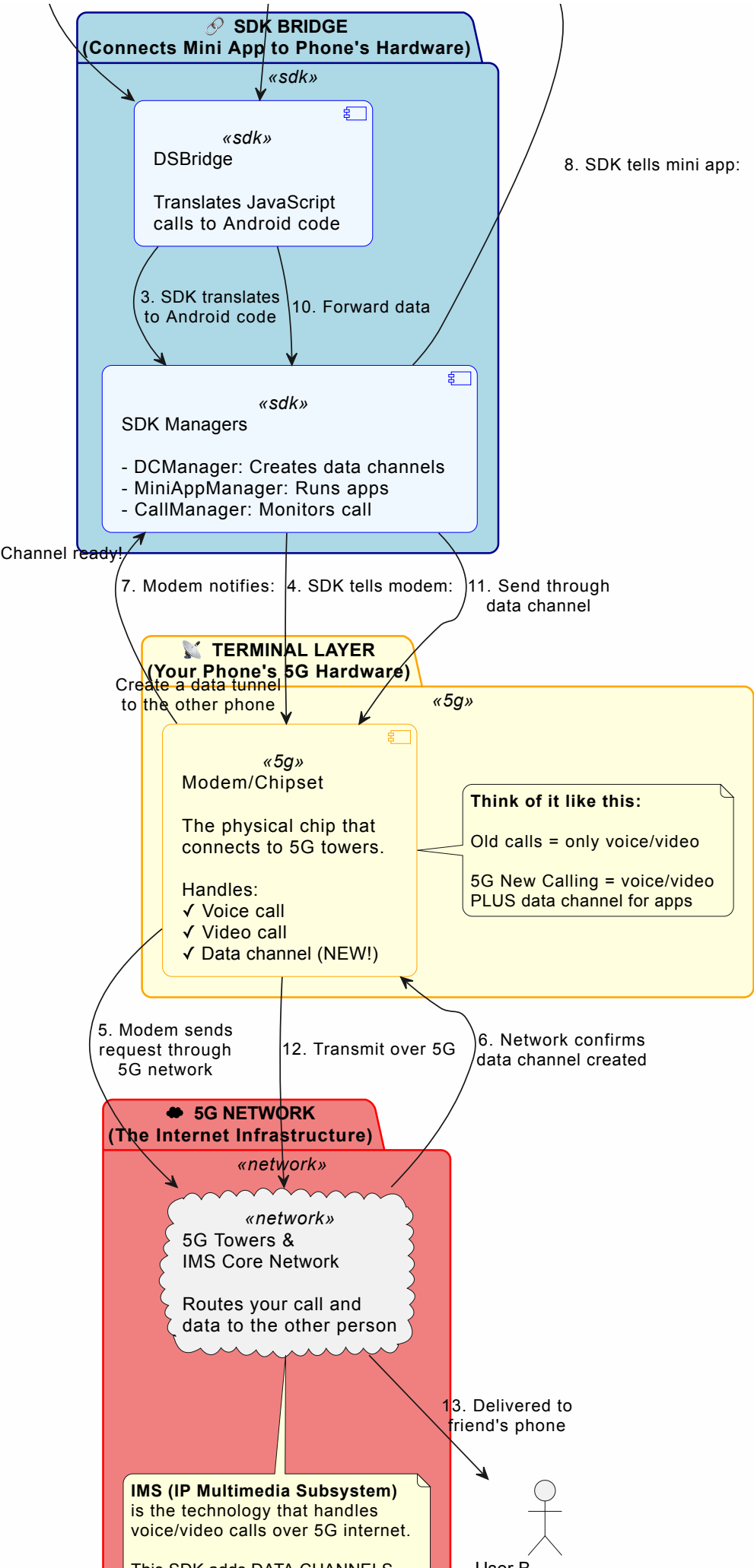
Alur Sistem Lengkap



Ringkasan Alur Sederhana

Simplified Flow: How Mini Apps Use Data Channels in 5G Calls





THIS SDK adds DATA CHANNELS on top of those calls.

User ID (Friend's Phone)

Concept	Simple Explanation
Mini App	Web app (HTML/JS) that runs during a call. Like a browser tab, but for calls.
SDK (This Project)	The software that lets mini apps talk to 5G hardware. Like a translator.
Data Channel	A "tunnel" for sending data during a call (separate from voice/video).
Terminal/Modem	The 5G chip in your phone. It talks to cell towers.
IMS Network	Internet-based phone system. Handles calls over 5G data (not old cellular).
AIDL	Android Interface Definition Language - how SDK talks to modem chip.

Proses Step-by-Step

Fase 1: Pembentukan Panggilan (Step 1-3)

1. 📞

User melakukan/menerima panggilan
 - InCallService mendeteksi panggilan aktif
 - SDK diinisialisasi
2. 🔍

SDK query mini app yang tersedia
 - AIDL call ke terminal
 - Terminal query IMS network
 - Mengembalikan daftar app (misal: "Menu Restoran", "File Share")
3. 📦

SDK download package mini app
 - AIDL download .zip dari network
 - Verifikasi signature & permission
 - Extract ke isolated storage

Fase 2: Peluncuran Mini App (Step 4-6)

4. 👉

User membuka mini app dari daftar
 - SDK membuat isolated WebView process
 - Load index.html dari package
 - Inject JSBridge untuk akses API
5. 🔗

Mini app request data channel
 - JavaScript memanggil: bridge.createDataChannel()
 - SDK validasi permission
 - Siapkan parameter channel
6. 📡

SDK request channel dari modem
 - AIDL call ke terminal
 - Modem negosiasi dengan network
 - WebRTC signaling membuat peer connection

Fase 3: Pertukaran Data (Step 7-9)

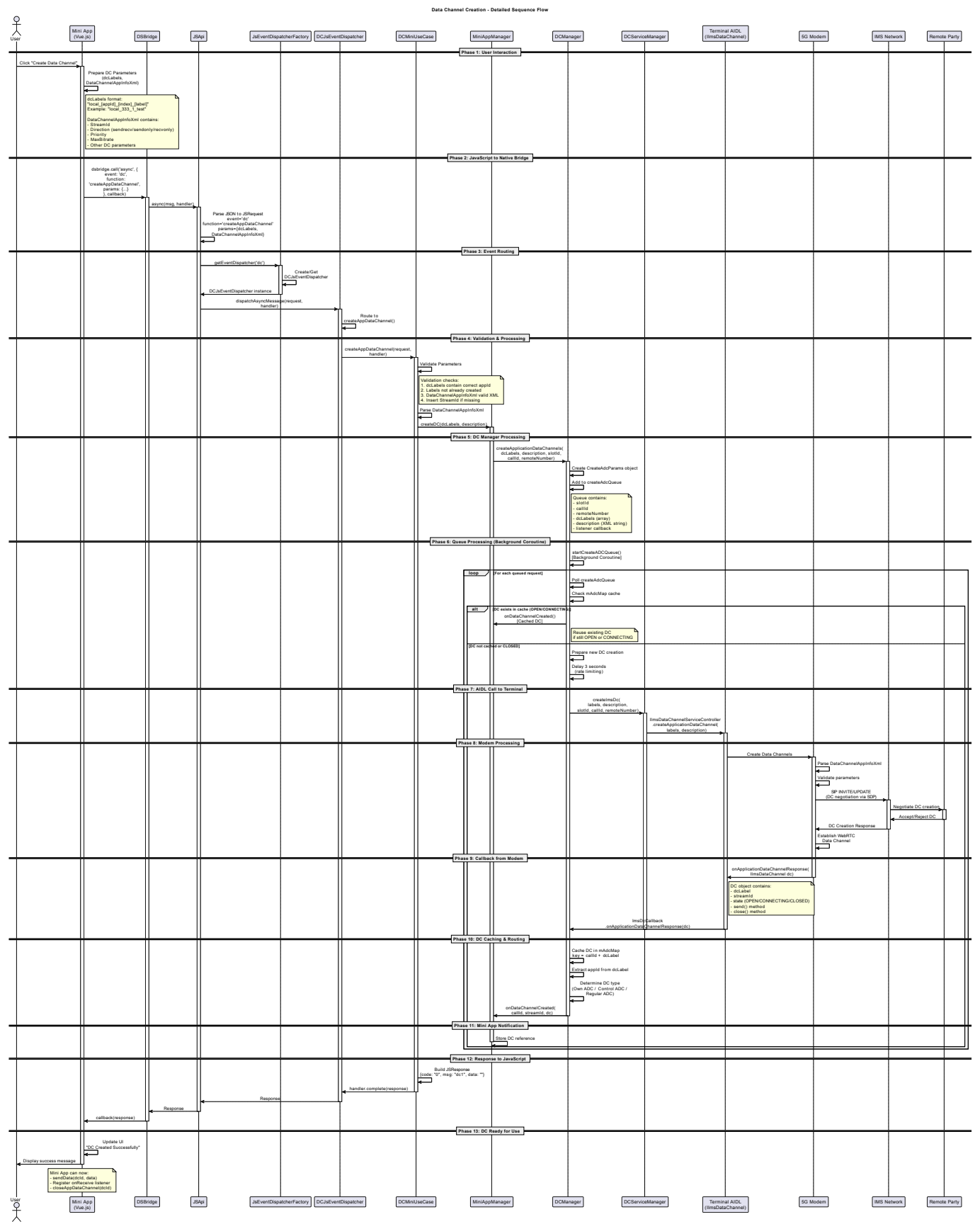
7. ✅ Pembuatan channel berhasil
 - Callback ke SDK dengan channel ID
 - SDK memberitahu mini app via JS callback
 - UI mini app update ke "Connected"
8. 📡 Mini app mengirim data
 - JavaScript: `bridge.sendData({ type: 'menu-order', item: 'pizza' })`
 - SDK marshal data ke AIDL
 - Modem mengirim via IMS data channel
 - Pihak remote menerima
9. 📡 Menerima data
 - Modem menerima data IMS
 - AIDL callback ke SDK
 - SDK dispatch ke mini app yang benar
 - JavaScript callback dipicu dengan data

Fase 4: Akhir Panggilan (Step 10-11)

10. 📞 Panggilan berakhir
 - InCallService mendeteksi disconnect
 - SDK memicu cleanup
11. 🧹 Pembersihan resource
 - Tutup semua data channel (AIDL)
 - Terminate process mini app
 - Hapus file temporary
 - SDK masuk ke idle state

Pembuatan Data Channel

Sequence Pembuatan Data Channel



Fase Pembuatan Detail

Fase 1-4: Inisiasi Request

Mini App (JS)
↓ createDataChannel({ channelId: 'app-channel' })

```
JSApi.async()
  ↓ Validasi parameter
DCJsEventDispatcher
  ↓ Route ke handler yang benar
DCMiniUseCase
  ↓ Validasi business logic
```

Fase 5-8: Negosiasi Channel

```
DCManager
  ↓ createApplicationDataChannels(params)
AIDL Interface
  ↓ IIMSDataChannel.createAppDataChannel()
Layer Terminal
  ↓ Negosiasi modem dengan network
IMS Network
  ↓ Membuat WebRTC connection
```

Fase 9-13: Callback Sukses

```
IMS Network
  ↓ Channel siap, mengembalikan channel ID
Layer Terminal
  ↓ AIDL callback
DCManager
  ↓ onApplicationDataChannelResponse()
DCMiniUseCase
  ↓ Update state internal
DCJsEventDispatcher
  ↓ Siapkan response JS
Mini App (JS)
  ↓ Success callback dipicu
```

Parameter Kunci

```
// Request pembuatan channel
data class CreateChannelParams(
    val channelId: String,           // Identifier unik
    val label: String,               // Nama yang dapat dibaca manusia
    val ordered: Boolean = true,     // Jaminan urutan message
    val maxRetransmits: Int? = null, // Setting reliability
    val protocol: String = "sctp"    // Protocol transport
)
```

Manajemen Lifecycle Panggilan

1 IDLE (Tidak Ada Panggilan Aktif)

2 INITIALIZING (Panggilan Terdeteksi)

12 / 29

- Inisialisasi manager (DCManager, MiniAppManager, dll.)
- Setup koneksi AIDL
- Siapkan direktori isolated storage

Transisi: → READY

3 READY (Tersedia untuk Mini App)

State: SDK berjalan, menunggu interaksi user

Aksi:

- Tampilkan launcher mini app (floating ball atau integrasi dialer)
- Dengarkan input user
- Monitor perubahan status panggilan

Resource: Manager aktif, AIDL terkoneksi

4 MINI_APP_RUNNING (App Aktif)

State: Satu atau lebih mini app berjalan

Aksi:

- Process WebView aktif
- Data channel mungkin terbuka
- Memproses JS API call
- Menangani transmisi data

Resource: WebView, data channel, event loop

5 CLEANUP (Panggilan Berakhir)

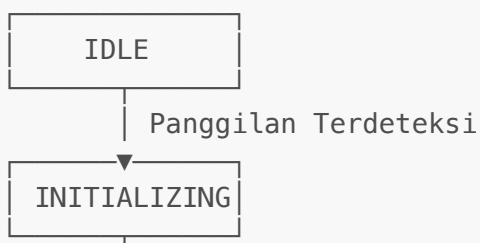
State: Panggilan terputus, membersihkan

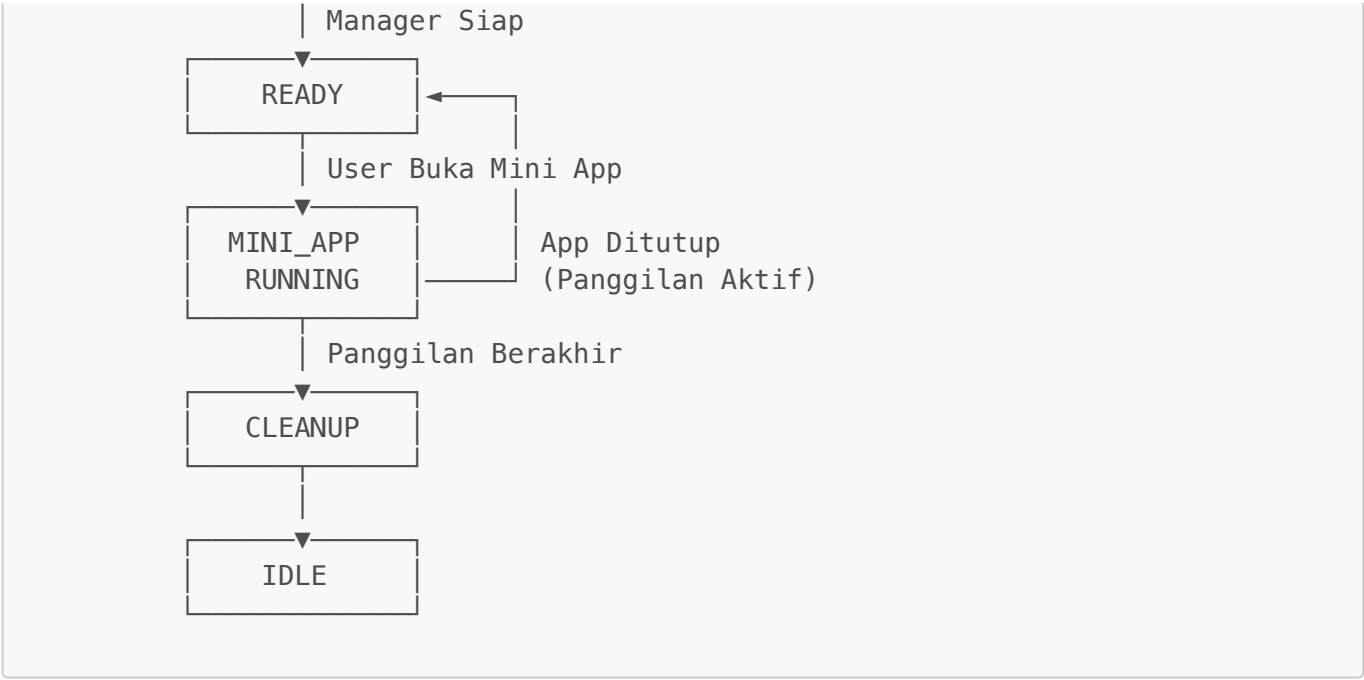
Aksi:

- InCallService.onCallRemoved() dipicu
- Tutup semua data channel
- Terminate process mini app
- Hapus cache (jika tidak dikonfigurasi persist)
- Lepas koneksi AIDL
- Dispose manager

Transisi: → IDLE

Diagram Transisi State





Struktur File & Organisasi Kode

Gambaran Struktur File



```

core/                                # 🔥 Logika utama SDK
├── aidl/                            # Definisi interface AIDL
│   ├── com/newcalllib/datachannel/V1_0/
│   │   ├── IImaDataChannel.aidl
│   │   ├── IImaDataChannelCallback.aidl
│   │   └── ...
├── core/
│   ├── common/                      # Utility
│   ├── constants/                  # Konstanta konfigurasi
│   ├── data/                       # Model data
│   ├── dispatcher/                 # Sistem routing event
│   │   ├── DCJsEventDispatcher.kt  # Route JS API call
│   │   └── MiniServiceEventDispatcher.kt
│   ├── factory/                    # Implementasi factory pattern
│   ├── manager/                    # Business logic inti
│   │   ├── call/
│   │   │   └── DCManager.kt        # Operasi data channel
│   │   ├── common/
│   │   │   └── ExpandingCapacityManager.kt # Manager extension
│   │   └── miniapp/
│   │       └── MiniAppManager.kt    # Lifecycle mini app
│   ├── miniapp/                    # Runtime mini app
│   │   ├── bridge/
│   │   │   └── JSApi.kt           # 🌟 Eksposur JavaScript API
│   │   ├── ui/
│   │   │   └── MiniAppWebView.kt  # Container WebView
│   │   └── ...
│   ├── service/                    # Service Android
│   │   └── InCallServiceImpl.kt    # 🌟 Monitor status panggilan
│   ├── usecase/                    # Handler business logic
│   │   ├── miniapp/
│   │   │   └── DCMiniUseCase.kt    # 🌟 Use case data channel
│   │   └── ...
│   └── utils/                       # Fungsi helper
└── libs/                           # Library third-party

miniapp/                            # Tool pengembangan mini app
├── webrtcDC/
│   └── webrtcDC.js                  # Library JavaScript untuk mini app
└── demo/                           # Contoh mini app
    ├── IMS_DC_Mini_app_demo_source_code/
    │   ├── index.html              # Entry point mini app
    │   ├── properties.json         # Metadata app
    │   └── src/                     # Source code Vue.js

```


oemec/	# Contoh extended capability OEM
└─ 0emEC.kt	# Implementasi EC contoh
script/	# Script build
testing/	# Tool simulasi lokal
document/	# 📖 Dokumentasi
└─ PROJECT_GUIDE.md	# Versi bahasa Inggris
└─ PROJECT_GUIDE_ID.md	# File ini (Bahasa Indonesia)
└─ START-HERE.md	# Panduan quick start
└─ *.svg	# Diagram arsitektur
└─ *.docx	# Spesifikasi API

File Kunci untuk Developer

Entry Point Inti

File	Tujuan	Baris yang Menarik
InCallServiceImpl.kt	Monitoring status panggilan	onCallAdded(), onCallRemoved()
JSApi.kt	Definisi JS API	Semua method @JavascriptInterface
DCManager.kt	Operasi data channel	createApplicationDataChannels() di baris 328
MiniAppManager.kt	Lifecycle mini app	launchMiniApp(), closeMiniApp()

Interface AIDL

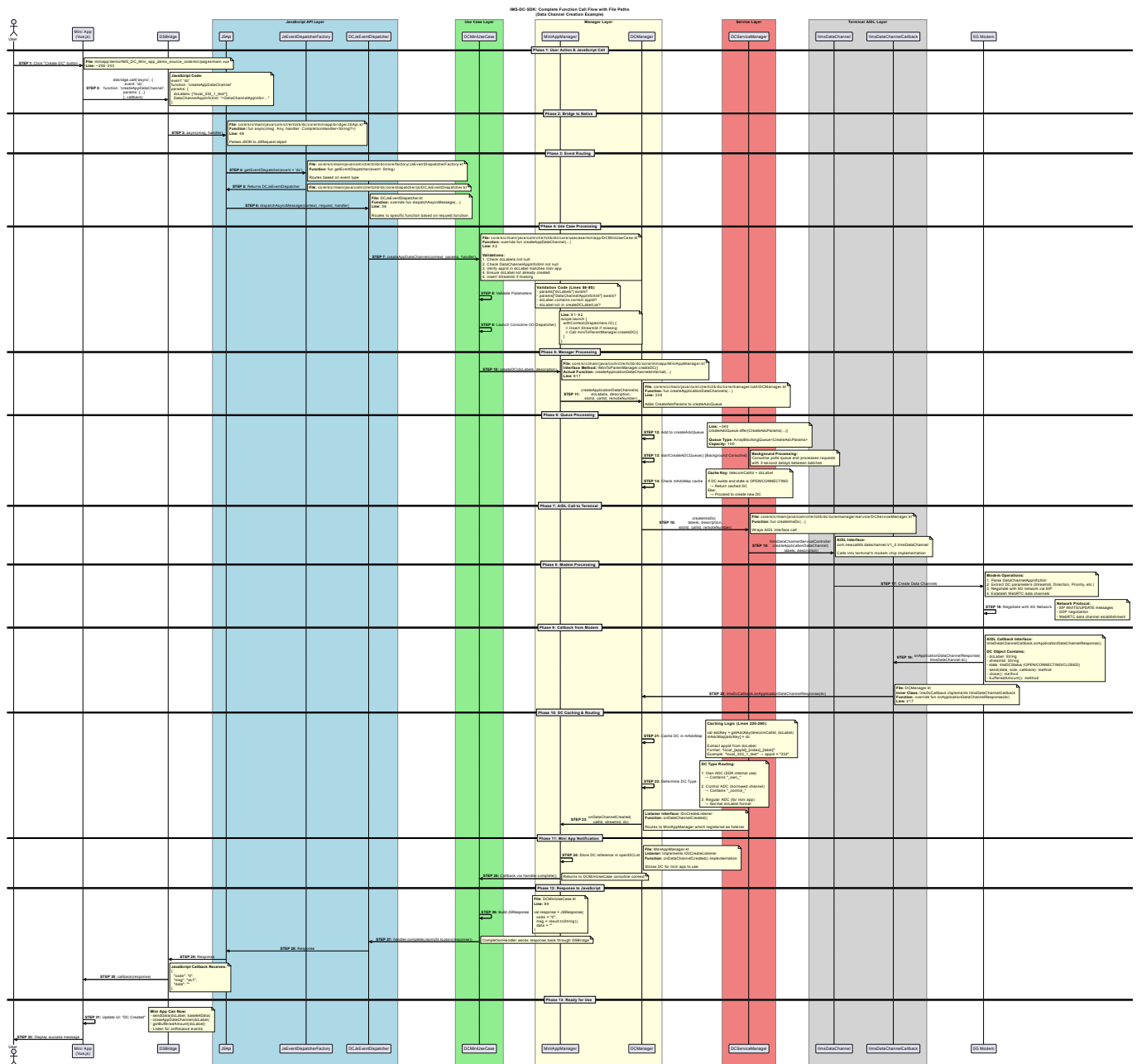
File	Tujuan
IImaDataChannel.aidl	Interface terminal untuk operasi DC
IImaDataChannelCallback.aidl	Callback dari terminal ke SDK

Use Case (Business Logic)

File	Tanggung Jawab
DCMiniUseCase.kt	Menangani request data channel dari mini app
MiniServiceEventUseCase.kt	Memproses event lifecycle mini app

Alur Fungsi Detail

Pembuatan Data Channel Level Fungsi



Alur Kode dengan Nomor Baris

Step 1-3: JavaScript Call → Entry SDK

```
// Kode mini app
bridge.async("createDataChannel", { channelId: "my-channel" },
function(response) {
    console.log("Channel dibuat:", response)
})
```

```
// JSApi.kt:48
@JavascriptInterface
fun async(msg: String, handler: CompletionHandler<String>) {
    // Parse JSON message
    val request = JSONObject(msg)
```

```
// Route ke dispatcher
dispatcher.dispatchAsyncMessage(request, handler)
}
```

Step 4-6: Event Dispatching

```
// DCJsEventDispatcher.kt:36
fun dispatchAsyncMessage(request: JSONObject, handler:
CompletionHandler<String>) {
    when (request.getString("method")) {
        "createDataChannel" -> {
            val params = request.getJSONObject("params")
            dcMiniUseCase.createAppDataChannel(params, handler)
        }
        // ... method lain
    }
}
```

Step 7-9: Logika Use Case

```
// DCMiniUseCase.kt:52
suspend fun createAppDataChannel(
    params: JSONObject,
    handler: CompletionHandler<String>
) {
    // 1. Validasi parameter
    val channelId = params.getString("channelId")
    require(channelId.isNotEmpty()) { "Channel ID diperlukan" }

    // 2. Cek permission
    if (!permissionManager.hasDataChannelPermission(currentMiniApp)) {
        handler.complete(createErrorResponse("Permission ditolak"))
        return
    }

    // 3. Forward ke manager
    dcManager.createApplicationDataChannels(
        channelId = channelId,
        label = params.optString("label", channelId),
        ordered = params.optBoolean("ordered", true),
        callback = { result ->
            handler.complete(result.toJSON())
        }
    )
}
```

Step 10-12: Manager → AIDL Call

```
// DCManger.kt:328
fun createApplicationDataChannels(
    channelId: String,
    label: String,
    ordered: Boolean,
    callback: (ChannelResult) -> Unit
) {
    // Simpan callback untuk nanti
    pendingCallbacks[channelId] = callback

    // Siapkan parameter AIDL
    val params = Bundle().apply {
        putString("channel_id", channelId)
        putString("label", label)
        putBoolean("ordered", ordered)
    }

    // Panggil terminal via AIDL
    try {
        imsDataChannelService?.createAppDataChannel(params, callbackBinder)
    } catch (e: RemoteException) {
        callback(ChannelResult.Error("AIDL call gagal"))
        pendingCallbacks.remove(channelId)
    }
}
```

Step 13: Pemrosesan AIDL Callback

```
// DCManger.kt:217 (callback handler)
private val callbackBinder = object : IImDataChannelCallback.Stub() {
    override fun onApplicationDataChannelResponse(
        channelId: String,
        result: Int,
        message: String
    ) {
        // Ambil callback yang disimpan
        val callback = pendingCallbacks.remove(channelId)

        // Proses result
        when (result) {
            RESULT_SUCCESS -> {
                callback?.invoke(ChannelResult.Success(channelId))
            }
            else -> {
                callback?.invoke(ChannelResult.Error(message))
            }
        }
    }
}
```

Referensi Fungsi Kunci

Komponen	File	Fungsi	Baris	Tujuan
Entry JS	JSApi.kt	<code>async()</code>	48	Menerima JS API call
Dispatcher	DCJsEventDispatcher.kt	<code>dispatchAsyncMessage()</code>	36	Route ke handler yang benar
Use Case	DCMiniUseCase.kt	<code>createAppDataChannel()</code>	52	Business logic & validasi
Manager	DCManager.kt	<code>createApplicationDataChannels()</code>	328	Komunikasi AIDL
Callback	DCManager.kt	<code>onApplicationDataChannelResponse()</code>	217	Menangani response AIDL

Panduan Pengembangan

Pengembangan SDK

Prasyarat

- **JDK:** Versi 17
- **Gradle:** Versi 8.1
- **Android SDK:** compileSdk 34, minSdk 26
- **IDE:** Android Studio (direkomendasikan)

Build SDK

```
# Clone repository
git clone <repository-url>
cd IMS-DC-SDK-Open-Source

# Build semua varian
./gradlew assembleRelease

# Output APK:
# - app/build/outputs/apk/normal/release/      # Entry floating ball
# - app/build/outputs/apk/dialer/release/      # Integrasi dialer
# - app/build/outputs/apk/local/release/       # Debugging lokal
```

Varian SDK

Varian	Entry Point	Use Case
--------	-------------	----------

Varian	Entry Point	Use Case
Normal	Floating ball saat panggilan	Distribusi umum
Dialer	Tombol dialer native	Integrasi OEM
Local	Icon app di home screen	Development & testing

Pengembangan Mini App

1. Setup Development Environment

```
# Gunakan demo sebagai template
cd miniapp/demo/IMS_DC_Mini_app_demo_source_code

# Install dependency
npm install

# Mulai development server
npm run dev
```

2. File yang Diperlukan

Setiap package mini app harus berisi:

index.html (Entry Point)

```
<!DOCTYPE html>
<html>
<head>
  <title>Mini App Saya</title>
  <script src="webrtcDC.js"></script>
</head>
<body>
  <div id="app"></div>
  <script src="main.js"></script>
</body>
</html>
```

properties.json (Metadata)

```
{
  "app_id": "com.example.myapp",
  "app_name": "Mini App Saya",
  "version": "1.0.0",
  "description": "Contoh mini app",
  "permissions": [
    "DATA_CHANNEL",
```

```

    "FILE_ACCESS"
  ],
  "icon": "icon.png",
  "entry": "index.html"
}

```

3. Menggunakan JS API

```

// Inisialisasi bridge
const bridge = window.dsBridge

// Buat data channel
bridge.async('createDataChannel', {
  channelId: 'my-channel',
  label: 'Channel Saya',
  ordered: true
}, function(response) {
  if (response.success) {
    console.log('Channel dibuat:', response.channelId)
  }
})

// Kirim data
bridge.async('sendData', {
  channelId: 'my-channel',
  data: JSON.stringify({ type: 'message', content: 'Halo!' })
}, function(response) {
  console.log('Data terkirim')
})

// Terima data (callback)
window.onDataReceived = function(channelId, data) {
  console.log('Diterima di', channelId, ':', data)
}

```

4. Package Mini App

```

# Build versi production
npm run build

# Struktur package:
# dist/
# |— index.html
# |— properties.json
# |— assets/
# |   |— js/
# |   |— css/
# |   |— images/
# |— icon.png

```

```
# Buat zip package
cd dist
zip -r ../MyMiniApp.zip *
```

5. Testing Lokal

```
# Install varian Local SDK ke device
adb install app-local-release.apk

# Push package mini app ke device
adb push MyMiniApp.zip /sdcard/

# Jalankan SDK
adb shell am start -n com.ct.ertclib.dc/.MainActivity

# Konfigurasi di app:
# Settings → Local Debugging → Load Package → /sdcard/MyMiniApp.zip
```

Adaptasi Terminal

Manufacturer terminal harus mengimplementasikan interface AIDL sesuai dengan **Spesifikasi Adaptasi Terminal SDK 5G New Calling** (lihat folder [document/](#)).

Poin Adaptasi Kunci

1. Implementasikan Interface AIDL TS.71

- [IImsDataChannel.aidl](#) - Operasi data channel
- Vendor harus menyediakan implementasi di system service

2. Integrasi dengan Modem/RIL

- Hubungkan AIDL call ke kapabilitas IMS DC modem
- Handle WebRTC signaling dengan network

3. Integrasi Sistem

- Daftarkan SDK sebagai InCallService default
- Sediakan integration point dialer (untuk varian Dialer)

4. Testing & Sertifikasi

- Verifikasi semua method AIDL bekerja dengan benar
- Test dengan mini app referensi
- Performance benchmarking

Sumber Daya Tambahan

File Dokumentasi

Dokumen	Deskripsi	Lokasi
START-HERE.md	Panduan quick start untuk pemula	document/START-HERE.md
DIAGRAMS_README.md	Cara melihat/edit diagram PlantUML	document/DIAGRAMS_README.md
aidl-vendor-implementation.md	Panduan untuk vendor terminal	document/aidl-vendor-implementation.md
Spesifikasi JS API	Referensi API lengkap (Word doc)	document/5G New Calling IMS Data Channel JS API.docx
Spec Adaptasi Terminal	Panduan integrasi OEM (Word doc)	document/5G New Calling SDK Terminal Adaptation Specification.docx

Contoh Kode

Contoh	Deskripsi	Lokasi
Demo Mini App	Contoh app Vue.js dengan semua fitur	miniapp/demo/IMS_DC_Mini_app_demo_source_code/
Library WebRTC DC	Library JavaScript compliant TS.66	miniapp/webrtcDC/
Extension OEM	Contoh implementasi extended capability	oemec/src/main/java/com/ct/oemec/OemEC.kt

Diagram Arsitektur (SVG)

Semua diagram tersedia di folder [document/](#) sebagai file [.svg](#) untuk kemudahan melihat di browser atau dokumentasi.

Diagram	File
Penjelasan Konsep 5G	5G Concepts Explained.svg
Gambaran Arsitektur	IMS-DC-SDK Architecture Overview.svg
Alur End-to-End	IMS-DC-SDK End-to-End Flow.svg
Ringkasan Alur Sederhana	Simplified IMS-DC-SDK Flow Summary.svg
Pembuatan Data Channel	Data Channel Creation Sequence.svg
Lifecycle Panggilan	Call Lifecycle Flow.svg
Struktur File	File Structure and Flow.svg
Fungsi Detail	Detailed Functions Flow – Data Channel Creation.svg

File Source PlantUML

Semua diagram dapat di-generate ulang dari file source [.puml](#) menggunakan:

- Online: <http://www.plantuml.com/plantuml/uml/>
 - VS Code: Extension PlantUML
 - IntelliJ/Android Studio: Plugin integrasi PlantUML
-

Standar & Spesifikasi

Standar yang Diimplementasikan

- **3GPP TS 26.114** - IMS Multimedia Telephony
- **GSMA TS.66** - WebRTC-based Data Channel API
- **GSMA TS.71** - AIDL Interface untuk IMS Data Channel
- **WebRTC** - Peer-to-peer data channel

Compliance

SDK ini dirancang untuk mematuhi standar telekomunikasi internasional sambil mendukung persyaratan khusus China Telecom untuk layanan 5G New Calling.

FAQ

Untuk Business/Product

T: Apa nilai bisnis dari SDK ini? J: Memungkinkan aliran pendapatan baru melalui layanan in-call: pemesanan makanan, peningkatan customer service, pengalaman AR, berbagi dokumen real-time, dll.

T: Siapa target user? J: Operator telekomunikasi, manufacturer perangkat (OEM), dan developer mini app yang membangun layanan untuk jaringan 5G.

T: Perangkat apa yang mendukung ini? J: Perangkat Android dengan modem 5G yang mengimplementasikan interface AIDL TS.71 (memerlukan adaptasi OEM).

Untuk Developer

T: Teknologi apa yang perlu saya ketahui? J:

- **Untuk pengembangan SDK:** Kotlin, Android, AIDL, Jetpack Compose
- **Untuk pengembangan mini app:** HTML5, CSS3, JavaScript (ES6+), dasar-dasar WebRTC

T: Bisakah saya test tanpa jaringan 5G? J: Ya! Gunakan varian "Local" yang mensimulasikan environment network untuk testing offline.

T: Bagaimana cara debug mini app? J: Gunakan Chrome DevTools dengan WebView debugging:

```
# Enable WebView debugging di SDK
adb shell setprop debug.webview.devtools true

# Buka chrome://inspect di browser Chrome
# Mini app Anda akan muncul di daftar
```

T: Apa perbedaan antara TS.66 dan TS.71? J:

- **TS.66** - Standar JavaScript API untuk mini app (berbasis WebRTC)
- **TS.71** - Standar interface AIDL antara SDK dan terminal

Untuk Manufacturer Terminal

T: Pekerjaan adaptasi apa yang diperlukan? J: Implementasi interface AIDL, integrasi dengan modem/RIL, registrasi system service. Lihat Spesifikasi Adaptasi Terminal.

T: Berapa lama waktu adaptasi? J: Biasanya 2-4 minggu untuk tim platform Android yang berpengalaman, termasuk testing.

T: Apakah sertifikasi diperlukan? J: Ya, China Telecom menyediakan testing sertifikasi untuk terminal yang sudah diadaptasi.

Kontak & Support

Tim Pengembangan

- **Email:** xuq17@chinatelecom.cn, pengc23@chinatelecom.cn
- **Organisasi:** China Telecom Research Institute

Berkontribusi

Ini adalah proyek open-source di bawah Apache 2.0 License. Kontribusi dipersilakan!

Pelaporan Issue

Untuk bug atau feature request, silakan hubungi tim pengembangan via email.

Lisensi

Copyright (C) China Telecom Research Institute

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

Appendix: Referensi Cepat

Method JS API Umum

Method	Parameter	Tujuan
<code>createDataChannel</code>	<code>{ channelId, label, ordered }</code>	Buat data channel baru
<code>sendData</code>	<code>{ channelId, data }</code>	Kirim data di channel
<code>closeDataChannel</code>	<code>{ channelId }</code>	Tutup data channel
<code>getChannelList</code>	<code>{}</code>	Dapatkan semua channel terbuka
<code>requestPermission</code>	<code>{ permission }</code>	Request runtime permission

Method Interface AIDL

Method	Tujuan	Callback
<code>createAppDataChannel()</code>	Buat channel	<code>onApplicationDataChannelResponse()</code>
<code>sendAppData()</code>	Kirim data	<code>onSendDataResponse()</code>
<code>closeAppDataChannel()</code>	Tutup channel	<code>onApplicationDataChannelClosed()</code>
<code>receiveAppData()</code>	N/A (callback saja)	<code>onReceiveData()</code>

Referensi Cepat File Path

 File SDK Kunci	
└─ InCallServiceImpl.kt	→
core/src/main/java/com/ct/ertclib/dc/core/service/	
└─ JSApi.kt	→
core/src/main/java/com/ct/ertclib/dc/core/miniapp/bridge/	
└─ DCManager.kt	→
core/src/main/java/com/ct/ertclib/dc/core/manager/call/	
└─ DCMiniUseCase.kt	→
core/src/main/java/com/ct/ertclib/dc/core/usecase/miniapp/	
└─ IImDataChannel.aidl	→
core/src/main/aidl/com/newcalllib/datachannel/V1_0/	
 Pengembangan Mini App	
└─ Source code demo	→
miniapp/demo/IMS_DC_Mini_app_demo_source_code/	
└─ Library WebRTC DC	→ miniapp/webrtcDC/
└─ Package contoh	→ miniapp/demo/*.zip
 Dokumentasi	
└─ Panduan ini	→ document/PROJECT_GUIDE_ID.md
└─ Versi English	→ document/PROJECT_GUIDE.md
└─ Quick start	→ document/START-HERE.md
└─ Diagram SVG	→ document/*.svg
└─ Source PlantUML	→ document/*.puml

Terakhir Diperbarui: 2026-02-13 **Versi Dokumen:** 1.0 **Versi SDK:** Lihat README.md